AI Application Engineering: 16-Week Job-Readiness Plan

Version 2.1 - corrected 8 August 2026

Outcome

Target roles: AI Application Engineer, GenAI Backend Engineer, AI Automation Engineer, and Backend Engineer - AI/LLM focus.

By 5 November 2026, publish one credible financial-document system that proves you can build, evaluate, secure, and operate an LLM application. The Coursera Professional Certificate is the learning spine; the existing financial earnings project is the portfolio proof.

Confirmed current checkpoint

- Calendar position: Week 4 of 16 (7-13 August 2026).
- Completed: Coursera Courses 1 and 2; Python/Pydantic baseline; direct Gemini structured extraction; basic evidence substring check; introductory LangChain and LlamaIndex RAG labs.
- Not started: Courses 3, 4, and 6. The learner was about to begin Course 6 but had not started it.
- Current single goal: complete Course 3 only.
- Project status: typed structured extraction and introductory chunk/prompt utilities exist. The main package does not yet contain embeddings, vector retrieval, reranking, evaluation, FastAPI, agent state, MCP, observability, or deployment.

Do not restart completed work. Do not begin Course 6 before the retrieval and service foundations.

Mandatory sequential operating model

The learner works on one focus block at a time. Course study and project implementation never run simultaneously.

Block	Exclusive focus	Exit condition
Course block	Complete one selected Coursera course; no project work	Course/module completion confirmed by the learner
Project sprint	Pause Coursera and implement the preceding course concepts in the existing financial project	Working code, tests/evals, and an honest project update
Portfolio block	Pause new course content and package evidence	Reproducible setup, results, architecture, limitations, and demo
Interview block	Pause feature expansion and practise explanations	Project deep dive and system-design answers can be defended

The sequence is:

```
Course 3 -> Retrieval sprint -> Course 4 -> Advanced-RAG sprint -> Service sprint ->
Course 6 -> Tools sprint -> Course 7 -> LangGraph sprint -> Course 9 -> MCP/security
sprint -> Portfolio -> Interviews
```

If a block takes longer than one calendar week, move the following dates. Never overlap two blocks to preserve the calendar.

Stack depth

Go deep: Python 3.12+, typing, Pydantic, pytest, FastAPI, one model provider, structured outputs, PostgreSQL with pgvector, hybrid retrieval, reranking, safe tool calling, LangGraph where state adds value, MCP security basics, OpenTelemetry-compatible tracing, Docker Compose, GitHub Actions, and one cloud deployment.

Know at working awareness: LangChain, LlamaIndex, Chroma, FAISS, multimodal parsing, self-hosted inference, fine-tuning, and multi-agent frameworks.

Defer unless a target job requires it: foundation-model training, elaborate multi-agent swarms, custom vector databases, GPU/Kubernetes operations, and a second polished project.

Weeks 1-3: completed foundation (17 July-6 August)

Evidence already produced:

- Typed Pydantic `Document`, `FinancialMetric`, and `EarningsExtraction` models.
- Unit tests for schema validation and fake extraction scenarios.
- Direct Gemini call using a response schema.
- Basic paragraph grouping and augmented prompt construction.
- Course 1 and 2 master notes plus LangChain/LlamaIndex practice scripts.

Carry-forward corrections:

- Structured output proves shape, not truth.
- A verbatim evidence match proves source presence, not claim support.
- Course scripts are practice artifacts; job-ready retrieval must live in the main package and be measured.

Week 4: Course 3 only (7-13 August)

Exclusive focus: Course 3 - vector databases, embeddings, vector operations, similarity search, and indexing.

Do:

- Complete the course modules and required labs.
- Write concise notes in your own words.
- Understand embeddings, cosine/dot/L2, exact versus approximate search, indexing, metadata filters, and recall@k.

Do not:

- Work on the financial project.
- Start Course 4 or Course 6.
- Build a second tutorial outside the required course work.

Exit check: the learner confirms Course 3 is complete and can explain how vector retrieval selects candidate evidence.

Week 5: Project Sprint A - retrieval foundation (14-20 August target)

Exclusive focus: the existing financial data/earnings project. Coursera is paused.

Build:

- Restore the development environment and run tests/lint reproducibly.
- Fix chunking so `chunk_size` and `chunk_overlap` are honored, including long paragraphs.
- Add stable document/chunk IDs and source/page/section metadata.
- Ingest two or three public earnings documents.
- Add embeddings and a persistent vector index under `src/`.
- Return ranked chunks and scores separately from generation.
- Create 15 retrieval questions with expected source chunks and report recall@k/MRR.

Exit check: retrieval can be run and evaluated without calling an LLM.

Week 6: Course 4 only (21-27 August target)

Exclusive focus: Course 4 - advanced retrievers and RAG patterns. No project work.

Learn:

- Advanced retrieval patterns and FAISS concepts.
- Precision/recall trade-offs, metadata filters, hybrid search, reranking, parent-child retrieval, and lost-in-the-middle.
- How retrieval failure differs from generation failure.

Exit check: Course 4 is complete and the learner can choose a retrieval strategy for a stated failure mode.

Week 7: Project Sprint B - advanced RAG and evaluation (28 August-3 September target)

Exclusive focus: improve the existing financial retrieval pipeline. Coursera is paused.

Build:

- Add sparse keyword retrieval and combine it with dense retrieval.
- Implement rank fusion and add a reranker only if evaluation shows value.
- Compare chunking and top-k configurations on the same dataset.
- Add stable citations, refusal behavior, and claim-to-evidence checks.
- Add conflict, missing-fact, duplicated-passage, unit/period, and indirect-prompt-injection cases.
- Measure retrieval and generation separately: recall/precision/MRR, answer correctness, citation support, refusal accuracy, schema success, latency, and cost.

Exit check: one measured retrieval configuration is selected for a documented reason.

Week 8: Project Sprint C - FastAPI and reliability (4-10 September target)

Exclusive focus: productionize the existing project. No Coursera work.

Build:

- Add typed health, ingest, ingestion-status, query, and extraction endpoints.
- Separate endpoint, domain, retrieval, storage, and model-provider code.
- Move long ingestion off the request path.
- Add timeouts, bounded retries, error mapping, request IDs, provider mocks, and API tests.
- Add Postgres/pgvector as the durable store; add Redis only for a measured job-state, limit, or cache need.
- Add structured logs and OpenTelemetry-compatible traces.

Exit check: core tests do not require a live model API, and one failed request is diagnosable.

Week 9: Course 6 only (11-17 September target)

Exclusive focus: Course 6 - function calling, chaining, tool orchestration, and agent fundamentals. No project work.

Learn:

- Tool schemas and manual tool-call handling.
- Model proposal versus application authorization.
- Side effects, idempotency, retries, timeouts, budgets, and auditability.
- When a deterministic workflow is safer than an agent.

Exit check: Course 6 is complete and the learner can explain why a model cannot authorize its own tool calls.

Week 10: Project Sprint D - safe tools (18-24 September target)

Exclusive focus: add tools to the financial project. Coursera is paused.

Build:

- Read-only document search, deterministic financial calculator, and metrics lookup tools.
- Pydantic tool arguments and application-side authorization/allowlists.
- Budgets for steps, time, tokens, and tool calls.
- Explicit terminal states, bounded retries, idempotency rules, and audit records.
- A direct Python workflow before any agent framework.

Exit check: invalid and unauthorized tool requests fail safely and are tested.

Week 11: Course 7 only (25 September-1 October target)

Exclusive focus: Course 7 - stateful workflows, ReAct, LangGraph, and agentic RAG. No project work.

Learn explicit state, routing, cycles, checkpointing, recovery, interrupts, termination, human approval, and replay.

Exit check: Course 7 is complete and the learner can distinguish a chain, deterministic workflow, and agent.

Week 12: Project Sprint E - LangGraph where justified (2-8 October target)

Exclusive focus: compare a graph with the existing direct Python workflow. Coursera is paused.

Build explicit states such as interpret, retrieve, calculate, draft, validate, approve, and finish. Add checkpointing only if recovery/resume is demonstrated. Keep LangGraph only where it improves control, recovery, or observability.

Exit check: the workflow is understandable from a state diagram and tests, including stopping and denied-action paths.

Week 13: Course 9 only (9-15 October target)

Exclusive focus: Course 9 - Model Context Protocol. No project work.

Learn MCP concepts plus least privilege, explicit consent, audience-bound tokens, secure token storage, no token passthrough, confused-deputy risks, malicious metadata, SSRF, exfiltration, and indirect prompt injection.

Course 8 remains optional breadth; do not insert it into the critical path unless a target job requires a specific framework.

Exit check: Course 9 is complete and the learner can explain what MCP standardizes and what security remains the host/application's responsibility.

Week 14: Project Sprint F - MCP, security, and deployment (16-22 October target)

Exclusive focus: finish and deploy the financial project. Coursera is paused.

Build:

- One least-privilege, read-only MCP capability with denied-case tests.
- A threat model mapped to current NIST and OWASP guidance.
- Red-team cases for injection, sensitive-data leakage, excessive agency, poisoning, and unbounded consumption.
- Dockerized API/dependencies with health checks and non-root execution.
- One cloud deployment with secrets, migrations, rollback, limits, and a failure runbook.
- Data provenance, retention/deletion, access control, human oversight, and EU AI Act awareness notes.

Exit check: the public demo is bounded, observable, recoverable, and has tested security boundaries.

Week 15: portfolio and application launch (23-29 October target)

Exclusive focus: package evidence and begin targeted applications. No new feature expansion.

Publish:

- Honest README and architecture diagram.
- Setup under 10 minutes and API examples.
- Evaluation method and before/after results.
- Threat model, operational runbook, and cost/latency table.
- Five-to-seven-minute walkthrough centered on one failure and its diagnosis.
- Resume bullets that quantify implemented scope/results without calling the prototype production-grade.

Week 16: interviews and gap closure (30 October-5 November target)

Exclusive focus: interview preparation.

- Run two project deep dives and two AI system-design mocks.
- Review Python, async/API design, SQL, queues/caches, and reliability.
- Practise RAG diagnosis, safe tools, evaluation, observability, deployment, and security answers.
- Fix only gaps exposed by target job descriptions or mock interviews.

Portfolio definition of done

- Real ingestion and retrieval live in `src/`, not only course scripts.
- Versioned evaluation data covers happy paths, refusals, adversarial inputs, and regressions.
- Retrieval and generation are measured separately.
- Citations use stable source identifiers and are checked for support.
- Model and tool boundaries use schemas, authorization, budgets, timeouts, and audit logs.
- FastAPI has mocked tests, reliable error handling, and background ingestion.
- Traces/logs support diagnosis without leaking secrets or sensitive content.
- Dockerized deployment, CI, rollback notes, threat model, and cost limits are documented.
- README distinguishes implemented features, measured results, and future work.

Job-readiness benchmark

You are ready to apply when you can:

1. Implement and diagnose a RAG pipeline end to end.
2. Explain retrieval metrics separately from answer quality.
3. Build a schema-validated extraction/API boundary and describe its failure modes.
4. Design safe tools with authorization outside the model and bounded execution.
5. Choose a deterministic workflow over an agent when dynamic planning is unnecessary.
6. Show traces, evaluation regressions, security tests, and one incident-style diagnosis.
7. Deploy and operate a Dockerized FastAPI service deliberately.

Primary standards and guidance

NIST GenAI Profile | OWASP LLM/GenAI Top 10 | MCP Security | OpenTelemetry GenAI | Google application lifecycle | Anthropic agent design | Anthropic agent evals | EU AI Act